Acta Informatica 34, 687–700 (1997)

© Springer-Verlag 1997

An analysis and implementation of an efficient in-place bucket sort

Apostolos Burnetas, Daniel Solow, Rishi Agarwal

Department of Operations Research, Case Western Reserve University, Cleveland, OH 44106, USA
(e-mail: dxs8@pyrite.som.cwru.edu)

Received 2 May 1995 / 31 July 1996

Abstract. Various methods, such as address-calculation sorts, distribution counting sorts, radix sorts, and bucket sorts, use the values of the numbers being sorted to increase efficiency but do so at the expense of requiring additional storage space. In this paper, a specific implementation of bucket sort is presented whose primary advantanges are that (i) linear average-time performance is achieved with an additional amount of storage equal to any fraction of the number of elements being sorted and (ii) no linked-list data structures are used (all sorting is done with arrays). Analytical and empirical results show the trade-off between the additional storage space used and the improved computational efficiency obtained. Computer simulations show that for lists containing 1,000 to 30,000 uniformly distributed positive integers, the sort developed here is faster than both Quicksort and a standard implementation of bucket sort. Furthermore, the running time increases with size at a slower rate.

1 Introduction

Because of its importance, sorting has been one of the most studied areas of computer science. On comparing the various methods that have been devised, two major approaches are evident: (1) the atomic comparison model, which assumes the the ordering of the elements can only be obtained by an element-to-element comparison and (2) the radix model in which the value of an individual element is used to estimate its location in the final sorted list. Many examples of both types of sorts are found in Knuth [11] and Cormen et al. [1].

Regardless of the model used, one can identify the following general considerations in designing a procedure for sorting n records:

1. Computational efficiency in both the theoretical and practical sense, including the ability:
   (a) To analyze and determine that the average running time, under appropriate assumptions on the probability distribution of the values sorted, is $O(n \log n)$ for atomic comparison-based methods and $O(n)$ for radix-based methods.
   (b) To mitigate worst-case behavior, should it arise, or to reduce the probability of its occurrence.
2. Conservation of storage space and the ability to control the additional amount used. An analysis of the space/time trade-off is useful in showing the cost/benefit of the additional storage space.

3. Ease of implementation and use. How difficult is the programming task? Can records stored in an array be sorted "in place" by exchanging records within the same array or is it necessary to use auxiliary data structures, such as linked lists?

In light of these criteria, no sorting procedure is perfect, not even Quicksort, which is considered to be the best available general-purpose, comparison-based sorting procedure, the details of which we assume the reader is familiar with. As shown first in Hoare [9] and subsequently in Sedgewick [13] for variations, the average running time of Quicksort is $O(n \log n)$, but its worst-case behavior is $O(n^2)$ (the median-of-three rule helps to avoid this situation). Because the sorting method proposed here is radix based, it has an average running time of $O(n)$; however, it does so without using linked lists (all sorting is done with arrays). This running time is achieved by using ρn additional storage, where ρ is any user-selected value between 0 and 1. (Computational results in Sect. 5 indicate that there is little additional benefit gained by making ρ larger than 0.2.)

To simplify the discussion that follows, assume that n records are to be sorted in increasing order based on keys that are stored as positive integers in the array $x[1], \ldots, x[n]$. In Sect. 4, an implementation of the proposed sort is discussed when the records contain multiple keys that need not be integers.

Various methods have been designed that use the values being sorted to increase efficiency, but do so at the expense of additional storage space. One of the first such methods is the address-calculation sort proposed by Isaac and Singleton [10]. The idea is to create a sorting function that associates to each key, an integer that corresponds to an approximate location of that record in the final sorted array. For example, when there are n keys whose values range from u to v, one can create the linear sorting function $mx + b$ through the two points $(u, 1)$ and (v, n). Then, for a given value of a key k, with $u \leq k \leq v$, the approximate location of that key in the sorted list is $f(k) = mk + b$ (rounded to the nearest integer). The idea, then, is to move each element $x[i]$ to its corresponding location in an output array, say, y, by setting $y[f(x[i])] := x[i]$. Unfortunately, f may not be one-to-one, so two different keys, $x[i]$ and $x[j]$, can result in $f(x[i]) = f(x[j])$. Because these keys cannot be stored in the same location, Isaac and Singleton describe a method for handling this problem that involves additional storage. No analytical results are obtained for the running time of this algorithm nor for the time/space trade-off.

Another example of an address-calculation sort is the *Franzi sort*, proposed by Suraweera and Al-Anzy in [15], in which the keys of the records range in value from u to v. Assuming that the values of the keys are unique, by using an additional array, say, $y[u..v]$ (whose values are set to 0 initially), each record $x[i]$ is moved directly from its original location to its correct location in y by setting $y[x[i]]$ to $x[i]$. The sorted list is then obtained by moving sequentially the nonzero elements of y back to the next available position of the original array x.

A related approach when faced with repeated key values, developed by Seward [14] and later by Feurzig [5], is to use the *distribution-counting sort*, described succinctly in Knuth [11]. This procedure requires the original array x of n elements, an output array z of size n for the sorted records, and an additional array – say, $Count$, of size $v - u + 1$ elements – to keep track of the number of times each value of a key occurs. Unfortunately, the user has no control over the values of u and v because they are based on the given keys. As a result, the amount of storage space used can be prohibitive. It is for this reason that Knuth remarks that these distribution-counting sorts are "primarily applicable in the case that many equal keys are present, and when

... ($v - u$) is small" and notes that this situation arises when sorting by the digits of the key, which is the basis of the *radix-list sort* (see Knuth [11]).

Another form of address calculation sort is bucket sort (see Devroye [2] for an overview), in which the range of the numbers being sorted is partitioned into K subranges. Each number in the list is then put into one of K corresponding groups called *buckets*, based on the subrange to which the number belongs. The numbers in each bucket are stored in a linked list to allow for an indeterminate number of values. In the sorts described in both Knuth [11] and Cormen et al. [1], the numbers in the buckets are sorted by insertion sort. The final sorted list is obtained by moving, in order, the numbers from each bucket to the output list. The advantage of bucket sorts is that numbers are moved quickly to their approximate location by being placed in the correct bucket. Under the assumption that the values being sorted are uniformly distributed throughout the range, the average number of elements in each bucket is approximately n/K which, it is hoped, can then be sorted efficiently by insertion sort. The disadvantages of this approach include: (1) the use of linked-list data structures, (2) the need for the additional storage for the n linked-list elements in the buckets together with K pointers to the head of each list, and (3) the fact that insertion sort can consume relatively large quantities of time if the number of elements in a bucket becomes excessive. This last disadvantage can be mitigated, to some degree, by using the more-efficient tree sort of Floyd [6] rather than insertion sort.

An early approach that eliminates the need for linked lists in bucket sort is developed in Gamson and Picard [7] where auxiliary arrays requiring $3n$ additional storage space are used. A subsequent improvement by Ducoin [4] reduced the additional storage requirements to $1.5n$. The *interpolation sort* – outlined in Gonnet [8] and based on interpolation search (see Knuth [11]) – requires only one auxiliary array of size k. However, Gonnet notes that the effort involved in using insertion sort to sort the k resulting sublists increases significantly as k decreases.

The specific implementation of an in-place bucket sort proposed here, called *Groupsort*, is a modification of the one in Ducoin. Groupsort retains the linear average running time for uniformly distributed numbers and the ability to sort in-place without using linked lists. The latter is accomplished by partitioning the array x into K subarrays – corresponding to the K buckets – and then moving the numbers from their original locations to the correct subarray, based on an interpolation function identical to the one used in interpolation search. The main differences between the implementation proposed here and the one in Ducoin are: (1) the reduction in additional storage from $1.5n$ to ρn, where ρ is any user-chosen value between 0 and 1 and (2) the use of Groupsort or Quicksort to sort the numbers in each bucket, thus improving on the use of insertion sort when the number of elements in a bucket is large.

The Groupsort algorithm is presented in Sect. 2 and is followed in Sect. 3 by an analysis of its average running time as well as a time/space trade-off. Extensions and other implementational issues are discussed in Sect. 4. In Sect. 5 empirical test results are shown to exceed the theoretical expectation obtained in Sect. 3.

2 Description of the proposed algorithm

The proposed bucket sort, called *Groupsort*, breaks the range of values, say, u to v, into K subranges of equal length, also referred to as *intervals*, numbered 0 through $K - 1$. (It is the flexibility in the choice of K that allows the user to control the

Values	75	32	199	98	1	125	137	88	170	25	165	188
Subscripts	1	2	3	4	5	6	7	8	9	10	11	12

a The original array to be sorted

Values	75	32	199	98	1	125	137	88	170	25	165	188
Subscripts	1	2	3	4	5	6	7	8	9	10	11	12
	↑			↑			↑		↑			
Group	0			1			2		3			

b The starting location of each subarray in the array

Values	1	32	25	75	98	88	125	137	199	170	165	188
Subscripts	1	2	3	4	5	6	7	8	9	10	11	12
	↑			↑			↑		↑			
Group	0			1			2		3			

c The list after all elements are moved to their groups

Fig. 1. An example of using Groupsort

amount of additional storage space that is needed subsequently.) Hereafter, the set of elements of the array being sorted that fall within each subrange is referred to as a *group*.

As mentioned previously, linked lists are used to store the numbers in the groups because it is not known, apriori, how many numbers are in each group. This problem is overcome here by using one additional pass through the array to count the number of elements that fall in each group. These totals are stored in an additional array $Count$ of K elements. Knowing the number of elements in each group allows one to partition the original array x into K subarrays (of varying lengths) that will eventually hold the numbers in each of the K groups, thus avoiding the need for linked lists.

To illustrate this feature, suppose there are 12 keys, stored in $x[1], \ldots, x[12]$, whose values range from u = 1 to v = 199, as shown in Fig. 1a. This range is then subdivded into, say, K = 4 equal intervals. In this case, the 4 intervals are:

Interval	Range of values
0	1 – 49
1	50 – 99
2	100 – 149
3	150 – 199

One additional pass through the array in Fig. 1a provides the number of elements (3, 3, 2, and 4, respectively) falling in each interval. This information results in the partition of the original array x into the 4 subarrays shown in Fig. 1b, in which the pointer (arrow) numbered k indicates the starting position for the numbers that will eventually be put in group k.

Having determined the starting location of each subarray, one more pass through the array is used to insert each element into the next available position of the group to which the element belongs. More specifically, for each $i = 1, \ldots, n$, $x[i]$ is exchanged

with the element in the next available position of the subarray in x to which $x[i]$ belongs. In this example, the first key, whose value is 75, belongs to group 1 and is therefore exchanged with the next available element in subarray 1 of the array x, namely, $x[4] = 98$, as seen in Fig. 1b. This exchange process eventually results in all numbers being placed in their appropriate group, as illustrated for this example in Fig. 1c. In the implementation that follows, it is shown how these steps are performed using one additional array of K elements, thus allowing the user to control the amount of additional storage space.

As with other bucket sorts, once the elements of x are moved to their proper group, the numbers in each group must be sorted to obtain the final list. Because in Groupsort the numbers in each group are stored in consecutive positions of the array, it is possible to use Quicksort to do so rather than the less efficient insertion sort.

Implementation

On the basis of the foregoing discussion, an efficient implementation of the proposed algorithm follows (in which it is assumed that the values of the n keys are stored as positive integers in the array elements $x[1], \ldots, x[n]$ and K groups are used):

An efficient implementation of Groupsort

Step 1: Find the minimum and maximum values of the keys, say, u and v, respectively, and move those elements to $x[1]$ and $x[n]$. It remains to sort only $x[2], \ldots, x[n-1]$.

Step 2: For each $j = 0, \ldots, K-1$, store into the array element $Count[j]$, the number of elements of x whose key has a value in interval j of the range u to v. That is, after initializing each $Count[j]$ to 0, compute, for each $i = 2, \ldots, n-1$,

$$j := \left\lfloor \frac{K * (x[i] - u)}{v - u + 1} \right\rfloor; \quad Count[j] := Count[j] + 1.$$

(The computation of j in this manner is the same as the corresponding computation in interpolation search.) In so doing, move any element belonging to group 0 or group $K-1$ to the next available position in those two subarrays (which is not possible for elements belonging to other groups). After this step, only those elements belonging to groups $1, \ldots, K-3$ must be moved. (The elements in group $K-2$ will then automatically be in their correct positions.)

Step 3: Now reuse the array $Count$ to indicate the starting location of each subarray by setting $Count[K-1]$ to $n + 1 - Count[K-1]$ and then computing, for each $j = K-2, \ldots, 1$,

$$Count[j] := Count[j+1] - Count[j].$$

When a number is moved to its correct group k, the value of $Count[k]$ is updated to keep track of the next available position in subarray k. Because the values of $Count$ are changing continuously, the position where one group ends and the next begins is lost. One way to maintain this information is to use a new array of K elements to mark the ending position of each group. However, it is possible to accomplish the same objective without using an additional array. This is done by converting the value in the last postition of each subarray into a sentinel value. When all keys are positive

integers, as is assumed here, a sentinel value is the negative of the last element in each group. These elements are made positive when they are moved. (To apply this technique to keys of other values, either an additional bit for each element or an additional array of K elements is needed.) In this case, therefore, mark the end of each subarray by replacing the element in the last position of each subarray with its negative.

Move each element $x[i]$, other than those in group 0 and group $K-1$, to the next available position in the subarray to which $x[i]$ belongs. This is done by moving sequentially all elements currently in subarray j ($j = 1, \ldots, K-3$) to their proper groups, until all elements in subarray j belong to group j. This, in turn, is accomplished by exchanging the element $x[i]$ in the next available position of subarray j with the element in the next available position of the subarray to which $x[i]$ belongs, until the element in $x[i]$ belongs to group j. The values of the array $Count$ that point to the next available position in each subarray are updated continuously. Negative values of the array elements indicate when one group is completed and the next group begins. These values are made positive when they are moved or when a group is completed. For further efficiency, before moving an element in the array, check to see if that element already is located in its correct group; if so, then skip that element and increase the pointer in that subarray by 1.

Step 4: Sort the numbers in each of the individual groups using Quicksort.

GroupSort is designed under the assumption that the values of the keys are distributed uniformly throughout the range. If this is not the case, the number of elements in a group can be large. When the number of elements in a group exceeds some critical value, the use of GroupSort, rather than QuickSort, to sort the numbers in that group may be more efficient. The determination of this critical value is given in the next section, together with an analysis of the average running time of the algorithm and a space/time trade-off.

3 Analysis of the Groupsort algorithm

In this section, the average running time and storage requirements of Groupsort are analyzed under the assumption of uniform distribution of the array elements. The objective is to determine, theoretically, (1) the trade-off between computation time and storage requirements for various choices of the number of groups as a function of the list length, and (2) how large the list must be for Groupsort to be more efficient than Quicksort, which will provide a guideline for choosing how to sort the groups in Step 4 of the algorithm.

Regarding the trade-off, the total processing time for sorting the list of n elements using Quicksort is denoted by $Q(n)$ and using Groupsort by $G(n)$. It is shown below that $\mathbf{E}G(n)$ is asymptotically decreasing in K_n, which makes a large number of groups desirable in terms of processing time. On the other hand, the additional storage space required by Groupsort, as compared to in–place sorting algorithms, is of order K_n, due to array *Count*. This makes a large number of groups undesirable in terms of storage.

Regarding the running time, the main result of the analysis is that, if the number of groups is selected as a fixed proportion of the list size, that is, $K = \rho n$, for some $\rho \in (0, 1)$, then the average running time of Groupsort is linear in n. A nonrigorous

justification of this result is as follows. If $K = \rho n$, the expected number of elements in each group is equal to $1/\rho$ (assuming that the elements are uniformly distributed). Then, the average time to sort each group is (approximately) $O((1/\rho)\log(1/\rho))$, and the average total time for Step 4 of the algorithm $n\log(1/\rho)$. The average time for Steps 1 to 3 is also linear in n, therefore, the total running time of Groupsort is $O(n)$. A formal justification of the running time, specially tailored for Groupsort, is given below. Conditions on the distribution of the list elements under which the average running time of standard versions of Bucketsort (using n buckets and linked-list data structures) is linear in n are given in Devroye and Klincsek [3].

In the implementation of Groupsort described in Sect. 2 it is assumed that the list elements are integers. This would make a discrete probability distribution model more appropriate. However, in order to avoid complications that would not alter the essence of the analysis, a continuous uniform distribution model is adopted for this section. From an implementation perspective, if the elements being sorted are real numbers, then the computation of the group number becomes

$$\left\lfloor \left(\frac{K}{v-u} \right) (x[i]-u) \right\rfloor ,$$

requiring one floating point multiplication (because $K/(v-u)$ need only be computed once). This group-number computation should result in slightly increased running times when compared to sorting an equivalent list of integers.

For the purposes of analysis, let $\underline{x} = \min_{i=1\ldots n} x[i]$ and $\overline{x} = \max_{i=1\ldots n} x[i]$. Then, given $\underline{x} = u$ and $\overline{x} = v$, $x[1], \ldots, x[n]$ are independent and identically distributed random variables following the uniform distribution on the interval $[u, v]$, that is,

$$P[x[i] \le x | \underline{x} = u,\ \overline{x} = v] = (x-u)/(v-u),\ \text{for } x \in [u, v].$$

This probabilistic model does not impose any restriction on the range of the elements because the assumptions of independence and uniform distribution are made *conditional* on the extreme values, which are not in any way restricted. Extensions to other distributions are discussed in Sect. 4.

The following notation is used in the sequel. For any $u, v \in \Re$ with $u < v$, $n = 1, 2, \ldots,$ and $K = K_n$ (allowed to depend on n), divide the range $[u, v]$ into K_n intervals (subranges) of equal length $(v-u)/K_n$. For $k = 0, \ldots, K_n - 1$, let I_k denote interval k, and S_k the size (cardinality) of the set $\{x[i], i = 1, \ldots, n :\ x[i] \in I_k\}$, that is the size of group k. Also let $\mathbf{P} = \mathbf{P}_{u,v}$, $\mathbf{E} = \mathbf{E}_{u,v}$ denote probability and expectation conditional on $\underline{x} = u$, $\overline{x} = v$.

From Sedgewick [13], the average running time of Quicksort for large n is approximately equal to

$$\mathbf{E}Q(n) \quad = \quad 11.667n\log n - 1.743n + 11.667\log n - 19. \tag{3.1}$$

For the analysis of the running time of Groupsort, the same conventions as in Sedgewick [13] regarding the times of elementary operations are followed, namely that "instructions which do not reference memory cost one time unit and instructions which do reference memory cost two time units."

Consider the implementation described in Sect. 2. The total running time is equal to $G(n) = G_1(n) + G_2(n) + G_3(n) + G_4(n)$, where $G_i(n)$ is the running time for Step i.

In Step 1, the maximum and minimum elements of array x are found. The operations performed for each element are at most two comparisons (3 time units

each) and one update (4 time units) of the extreme value and its position. Therefore, $\mathbf{E}G_1(n) \leq 10n$.

In Step 2, array *Count* is initialized, the group numbers of the elements of x are computed and the entries of *Count* are updated. In addition, for the nonempty groups the appropriate elements of x are converted into the sentinel values. Combining the above, $\mathbf{E}G_2(n) = 8n + 6K_n + 16\mathbf{E}F_n$, where F_n is the number of nonempty groups. Therefore, $\mathbf{E}G_2(n) \leq 8n + 22K_n$.

In Step 3, the group number of each element of x is computed and compared to the number of the group currently being processed. Every element not initially in the correct group is exchanged with the first not–in–place element in the destination group, and the corresponding entry of *Count* is updated. Combining the above, $\mathbf{E}G_3(n) \leq 20n$.

In Step 4, Quicksort is used for sorting each group; therefore, $\mathbf{E}G_4(n) = \sum_{k=0}^{K_n-1} \mathbf{E}Q(S_k)$. In Proposition 1, it is shown that the group sizes, S_k, are identically distributed for all k, and an upper bound for $\mathbf{E}G_4(n)$ is developed.

The following inequality from probability (c.f. Ross [12]) is used in the proof of Proposition 1.

Lemma 1 (Jensen's inequality) *Let X be a random variable with distribution $F(x)$ and let E denote expectation with respect to F. If $g(x), h(x)$ are convex and concave functions of x, respectively, then $E\,g(X) \geq g(E\,X)$ and $E\,h(X) \leq h(E\,X)$.*

Proposition 1

1. *For any $u, v \in \Re$ with $u < v$, and all $k = 0, \ldots, K_n - 1$, the conditional distribution of S_k given u, v is binomial with parameters n and $p_n = 1/K_n$.*
2. $$\mathbf{E}(\log S_0) \leq \log\left(\frac{n}{K_n}\right), \tag{3.2}$$

   and

   $$\frac{n}{K_n}\log\left(\frac{n}{K_n}\right) \leq \mathbf{E}(S_0 \log S_0) \leq \frac{n}{K_n}\log\left(\frac{n}{K_n}\right) + \left(1 - \frac{1}{K_n}\right). \tag{3.3}$$

3. *For large n,*

$$\begin{aligned} \mathbf{E}G_4(n) &= K_n \mathbf{E}Q(S_0) \\ &\leq 11.667\left(n\log\left(\frac{n}{K_n}\right) + K_n - 1\right) - 1.743n + \\ &\quad 11.667K_n \log\left(\frac{n}{K_n}\right) - 19K_n. \end{aligned} \tag{3.4}$$

Proof. (1) Let $u, v \in \Re$ with $u < v$. Since all intervals I_k have equal length, $P[x[i] \in I_k] = p_n = 1/K_n,\ \forall i = 1, \ldots, n,\ k = 0, \ldots, K_n - 1$. Since in addition $x[1], \ldots, x[n]$ are independent (given u, v), the joint conditional distribution of $S_0, \ldots, S_{K_n-1}$ is multinomial with parameters n and $\underline{p} = (p_n, \ldots, p_n)$. Thus, the (marginal) distribution of S_k is binomial with parameters n, p_n, for all $k = 0, \ldots, K_n - 1$. This proves part 1.

(2) Since $\log(S_0)$ is concave in S_0, (3.2) follows from Lemma 1, with $\mathbf{E}S_0 = n/K_n$. Since $S_0 \log(S_0)$ is convex in S_0, the first part of (3.3) follows from Lemma 1. Also, by part 1,

$$\begin{aligned}\mathbf{E}(S_0 \log S_0) &= \sum_{j=1}^{n} j \log j \binom{n}{j} p_n^j (1-p_n)^{n-j} \\ &= n\,p_n \sum_{j=0}^{n-1} \log(j+1) \binom{n-1}{j} p_n^j (1-p_n)^{n-1-j} \\ &= n\,p_n\,\mathbf{E}\log(Y+1),\end{aligned}$$

where Y represents a binomial random variable with parameters $n-1$ and p_n. Since $\log(Y+1)$ is concave in Y, it follows from Lemma 1 that

$$\begin{aligned}\mathbf{E}\log(Y+1) &\le \log(\mathbf{E}Y+1) = \log((n-1)p_n+1) \\ &= \log(np_n) + \log(1+(1-p_n)/(np_n)) \\ &\le \log(np_n) + (1-p_n)/(np_n),\end{aligned}$$

thus, the second inequality in (3.3) is shown.
(3) Because the group sizes, S_k, are identically distributed for all k, $\mathbf{E}G_4(n) = K_n\mathbf{E}Q(S_0)$. Also, from (3.1),

$$\mathbf{E}Q(S_0) = 11.667\mathbf{E}(S_0 \log S_0) - 1.743\mathbf{E}S_0 + 11.667\mathbf{E}(\log S_0) - 19.$$

Now the desired result follows using (3.2) and (3.3).

In the next Proposition it is shown that, in the special case where the number of groups is a fixed proportion of the list size n, the average running time of Groupsort is linear in n. This is achieved at the expense of ρn amount of additional storage space.

Proposition 2 *If* $K_n = \lfloor \rho n \rfloor$, *for some* $\rho \in (0,1)$, *then , for* $n \ge 2/\rho$, $\mathbf{E}G(n) \le n\,W_1(\rho) + W_2(\rho)$, *where*

$$\begin{aligned}W_1(\rho) &= 11.667(1+\rho)\log(1/\rho) + 14.667\rho + 36.257, \\ W_2(\rho) &= 23.334(1+\rho)/\rho + 7.333.\end{aligned}$$

Proof. Because $\rho n - 1 < K_n \le \rho n$, and $\log K_n > \log(\rho n - 1) = \log(\rho n) + \log(1 - 1/(\rho n))$, it follows from (3.4) that

$$\begin{aligned}\mathbf{E}G_4(n) \le\ & 11.667(n\log(1/\rho) - n\log(1-(1/(\rho n)) + \rho n - 1) - 1.743n + \\ & 11.667\rho n\log(1\rho) - \rho n\log(1-1/(\rho n)) - 19(\rho n - 1) \\ =\ & (11.667(1+\rho)\log(1/\rho) - 1.743 - 7.333\rho)n - \\ & 11.667(1+\rho)n\log(1-1/(\rho n)) + 7.333. \qquad (3.5)\end{aligned}$$

The proof follows combining (3.5) with the bounds for $\mathbf{E}G_i(n), i = 1,2,3,$ and observing that $-n\log(1-1/(\rho n)) = n\log(\rho n/(\rho n-1)) = n\log(1+1/(\rho n-1)) \le n/(\rho n-1) \le 2/\rho$, for $n \ge 2/\rho$.

From Proposition 2 it follows that there exists $n_0 = n_0(\rho)$, such that $\mathbf{E}G(n) \le \mathbf{E}Q(n)$, for $n \ge n_0$. So, n_0 is the minimum required array size for Groupsort to be faster than Quicksort. Also, $n_0 \le \overline{n_0}$, where $\overline{n_0}$ is the solution of $W_1(\rho)n + W_2(\rho) = \mathbf{E}Q(n)$ for n. Table 1 contains the values of $\overline{n_0}$ for several values of ρ.

The results of Proposition 1 also hold for any choice of K_n other than those proportional to n. For instance, if $K_n = \lfloor n^\alpha \rfloor$, for some $\alpha \in (0,1)$, then it can be

Table 1. Values of cutoff list size $\overline{n_0}$

ρ	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
$\overline{n_0}$	389	240	187	160	142	128	118	109	101

shown, similarly to Proposition 2, that $\mathbf{E}G(n) \sim (1-\alpha)\mathbf{E}Q(n)$, as $n \to \infty$. Hence, in this case, the running time of Groupsort is asymptotically equal to a fraction $1-\alpha$ of $\mathbf{E}Q(n)$, whereas the additional storage requirements are increasing at a (moderate) rate of n^{α}. This or some similar choice can be used when the additional storage space must be kept very low.

4 Other distributions and extensions

The uniform distribution assumption can be relaxed without altering the main analysis when the conditional distribution of the values of $x[i]$ is continuous and known a priori. In this case, the subranges I_k are defined (not necessarily of equal length) so that $\mathbf{P}[x[i] \in I_k] = 1/K_n$, for all $k = 0, \ldots, n-1$.

If, on the other hand, the distribution is discrete (which includes the case of integer key values), then the theoretical analysis in Sect. 3 holds approximately for large values of n if the range is sufficiently large so that the probability of an integer falling in each group is approximately the same. When the range is not sufficiently large, it may not be possible to achieve the linear average-time performance obtained in Sect. 3. For example, consider sorting $n = 10,000$ records based on a year field whose values range from $u = 1980$ to $v = 1989$. To achieve linear average-time performance, one should choose the number of groups to be, say, $K = 0.1n = 1000$. The result in this case is that group numbers 10 through 999 will be empty, so the effective number of groups is 10, resulting in a large number of elements in some groups. Observe that because the range $v-u+1$ is very small, one should in fact use the more efficient distribution counting sort. This raises the question of the effect of the range on the choice of the number of groups used in Groupsort.

In view of the foregoing discussion, an appropriate choice for the number of groups when the values to be sorted are integers is

$$K = \min\{v-u+1, \rho n\}, \tag{4.6}$$

for an appropriate value of ρ between 0 and 1. This observation also provides insight as to how to use Groupsort when sorting records on multiple fields. For example, consider sorting a list of $n = 10,000$ last names consisting of, say, $m = 20$ lowercase letters. One possibility is to create $K = 26$ groups, one for each of the 26 first letters in each name. This choice is suboptimal from the theoretical analysis because the average number of names in most groups is large.

A more desirable choice is, for example, $K = 0.1n = 1000$ groups. To use 1000 groups effectively, from (4.6), the range must exceed 1000. One way to arrange for this to happen is to create, for each last name, an integer in the range of 1 to 26^m, based on the m letters in the name. Specifically, if the individual letters in one last name are stored in the array $l[1], \ldots, l[m]$, then the associated integer is the following radix-26 positional notation (c.f. Knuth [11]):

$$\sum_{i=1}^{m} 26^{m-i}[y(l[i]) - y('a')], \tag{4.7}$$

where, for a letter ch, $y(ch)$ is the integer in the collating sequence corresponding to ch. One disadvantage of this approach is that even relatively small values of m can result in a range that exceeds the largest integer on the computer. Furthermore, the use of all m letters to convert a last name to an integer is time consuming.

A reasonable compromise, based on (4.6) and (4.7), is to use only as many letters in the last name as to insure that the range exceeds ρn. In this example, using only the first

$$m' = \left\lfloor \frac{\ln 1000}{\ln 26} \right\rfloor + 1 = 3$$

letters provides a range of 1 to $26^3 = 17576$, which can then be divided into $K = 1000$ groups.

Regardless of the distribution and the number of groups used, it is possible that the number of elements in any one of these groups is large (up to n, in the worst case). To mitigate this worst-case behavior, on the basis of the theoretical analysis in Sect. 3, it is advisable to use Groupsort to sort those sublists that have more than, say, 1000 elements.

5 Computational results for Groupsort

This section contains computational results for the running time of Groupsort, Quicksort and Bucketsort. The tests are performed on a personal computer with an INTEL 486-DX66 CPU. The Groupsort code was written in Borland Turbo C and does not call Groupsort recursively. The C function *qsort* was used for the implementation of Quicksort. According to their documentation, this implemenation uses the "median of three" rule for choosing the pivot element.

Regarding the computations of the group numbers in Steps 2 and 3, the following modification was used instead of the formula presented in Sect. 2:

$$j := \left\lfloor \frac{(x[i] - u)}{M} \right\rfloor, \text{ where } M = \left\lfloor \frac{v - u + 1}{K_n} \right\rfloor + 1.$$

The reason for this modification is that the formula in Sect. 2 involves either a floating point or a long integer division, both of which were found to increase significantly the running times in the simulation.

In the first series, Groupsort is tested against Quicksort and Bucketsort using a uniform distribution of integer array values in the range 0 to 32767, for list sizes n varying from 1,000 to 25,000. For smaller sizes, all methods require negligible times. A number of groups $K_n = \rho n$ is used for Groupsort and Bucketsort, with values of $\rho = 0.1, 0.2,$ and 0.3. The numbers in each bucket of Bucketsort are sorted with insertion sort (treesort (c.f. [6]) might improve the running time). For each value of n and ρ, 30 arrays are generated and sorted. Table 2 shows the average running times in milliseconds. These results are also presented in Fig. 2 for $\rho = 0.1$.

The results indicate that for all values $n \geq 1000$, Groupsort is faster than Quicksort, thus verifying that the cutoff point obtained in Sect. 3 is below 1000. These test also show that Groupsort is more efficient than Bucketsort, while requiring less storage and no linked-list data structures.

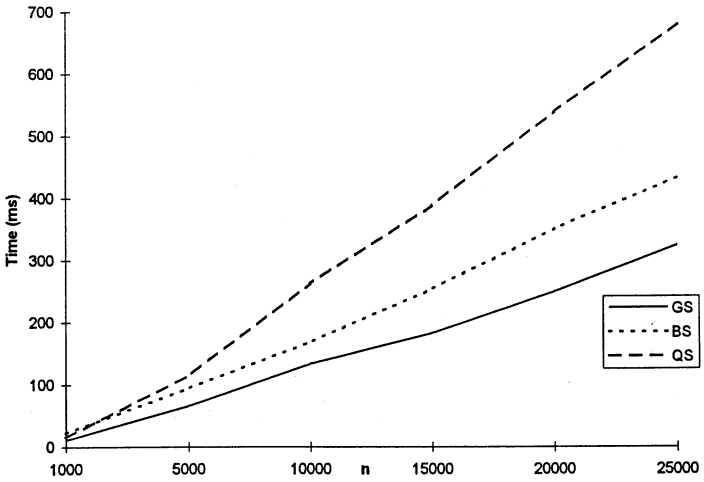


Fig. 2. Running times for uniform distribution with $K = 0.1n$

Table 2. Running times under a uniform distribution

ρ	0.1			0.2			0.3		
n	GS	BS	QS	GS	BS	QS	GS	BS	QS
1000	11	22	16	16	9	21	19	16	25
5000	66	94	115	56	88	117	57	72	123
10000	134	169	264	124	161	251	110	158	247
15000	183	255	388	176	238	395	172	237	402
20000	250	351	539	232	330	545	227	318	537
25000	325	434	681	301	405	680			

The second series of tests is designed to determine how the efficiency of Groupsort is affected by a non-uniform distribution of values, in comparison to Quicksort. To this end, the array elements are generated according to a normal distribution truncated in the range $0 - 32768$, with mean $\mu = 16384$ and standard deviation σ varying from 2 to 15000. Smaller values of σ imply a higher degree of nonuniformity. The average time (in milliseconds) of sorting 15 arrays of each list size and standard deviation is reported in Table 3 (see Fig. 3 for the case of $n = 20,000$). For each value of n in Table 3, Groupsort is faster than Quicksort when $\sigma = 1000, 10000$ and 15000, and the opposite is true when $\sigma = 2$ and 10, thus indicating that the numbers must be "sufficiently uniform" for the effect of sorting the (smaller) buckets in Step 4 of Groupsort to outweigh the additional overhead of the first three steps, in comparison to Quicksort. On the basis of these results one might conjecture that, for all sufficiently large n, there is a $\overline{\sigma}$ such that, for $\sigma > \overline{\sigma}$, Groupsort is more efficient than Quicksort. This, however, remains an open question.

Conclusion

An efficient in-place bucket sort is developed, analyzed, and tested. This algorithm, called Groupsort, breaks the range of key values into K equal parts and then moves each record into a portion of the array corresponding to the elements in that group.

Table 3. Running times under truncated normal distribution

n	σ	2	10	1000	10000	15000
1000	GS	11	15	15	19	15
	QS	0	15	22	22	21
5000	GS	57	87	84	77	77
	QS	36	59	114	131	123
10000	GS	117	143	170	144	150
	QS	78	114	235	270	270
20000	GS	202	275	309	305	296
	QS	135	227	491	569	565
30000	GS	319	401	447	442	451
	QS	267	353	743	894	847

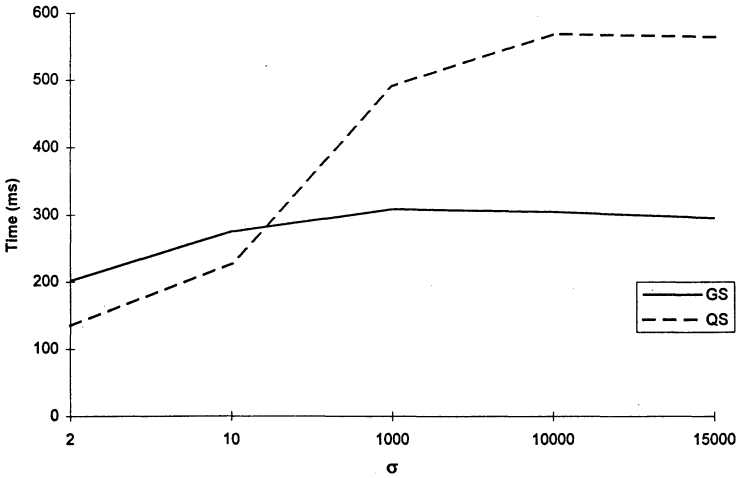


Fig. 3. Running times for normal distribution with $n = 20,000$

The primary advantage of this approach is that linear average-time performance is achieved, without linked lists, by using an additional amount of storage $K = \rho n$, for any value of ρ with $0 < \rho < 1$. Furthermore, the user has control over K.

As expected, Groupsort is faster than Quicksort, both theoretically and empirically, because the former uses the representation of the values being sorted, while the latter is an atomic comparison-based sort. In comparison to Bucketsort, which also uses the representation of the values, computational tests indicate that, on average, for uniformly-distributed random lists, Groupsort is faster for values of n with $1000 \leq n \leq 30,000$. The additional storage required for this increased efficiency is a single array of size as little as $0.10n$, while Bucketsort requires an additional n linked-list elements.

Acknowledgement. We are grateful to one of the anonymous referees whose detailed comments led to a more specific comparison of our algorithm with bucket sorts and other related methods.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L.: Introduction to algorithms. MIT Press, 1990
2. Devroye, L.: Lecture notes on bucket algorithms. Birkhauser, 1986
3. Devroye, L., and Klincsek, T.: Average time behavior of distributive sorting algorithms. Computing **26** (1981), 1–7
4. Ducoin, F.: Tri par addressage direct. R.A.I.R.O. Informatique/Computer Science **13**, 3 (1979), 225–237
5. Feurzig, W.: Algorithm 23, math sort. Commun. ACM **3** (1960), 601–602
6. Floyd, R. W.: Algorithm 245: Treesort 3. Commun. ACM **7**, 12 (1964), 701
7. Gamson, E., Picard, C.: Algorithme de tri par addressage direct. C. R. Acad. Sc. Paris 269, série A (1969), 38–41
8. Gonnet, G. H.: Handbook of Algorithms and Data Structures. Addison-Wesley, 1984
9. Hoare, C. A. R.: Quicksort. Comput. J. **5**, 4 (1962), 10–15
10. Isaac, E. J., Singleton, R. C.: Sorting by address calculation. J. ACM **3** (1956), 169–174
11. Knuth, D.: The Art of Computer Programming, vol. 3. 1979
12. Ross, S. M.: A First Course in Probability, 4th edn. Macmillan, 1994
13. Sedgewick, R.: The analysis of quicksort programs. Acta Informatica **7** (1977), 327–355
14. Seward, H. H.: Information sorting in the application of electronic digital computers to business operations. Tech. rep., MIT Digital Computer Laboratory, Report R-232, Cambridge, Mass, 1954
15. Suraweera, F., Al-Anzy, J. M.: Analysis of a modified address calculation sorting algorithm. Comput. J. **31**, 6 (1988), 561–563